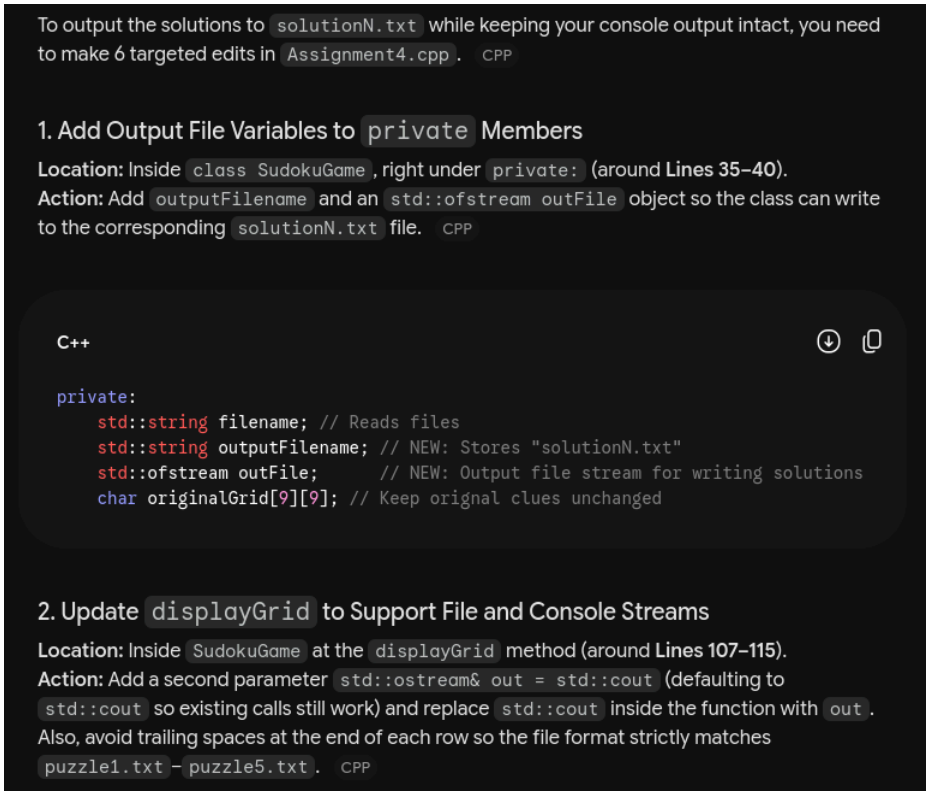
1. Gemini 4 Argon Medium and Qwen 3.8-27B (Reasoning Budget 1024)
2. Gemini via https://gemini.google.com/app, Qwen via LM Studio
3. (attached source as a file)

Given this code, modify it with the following requirements. Do not just output a modified version, but please point out where in the file you made changes, as I have to make changes to specific parts rather than replace the whole file:
Write the solutions for each puzzle file to a file named solutionN.txt, where N is the puzzle number.
* Each solutionN.txt file should include the following:
- If no solutions are found, a single line: “No solution found.”
- If solutions are found, it should include all solutions followed by a single line: “Solutions found: n”, where n is the number of solutions found.
- The solutions should be in the same format as the input files: puzzle1.txt – puzzle5.txt.

4.
    a. Gemini



Replace this:

C++

```cpp
// Helper method to print any 9x9 grid
void displayGrid(char grid[9][9]) {
    for (int i = 0; i < 9; i++) { // This for loop iterates through all 9 rows o
        for (int j = 0; j < 9; j++) { // It will iterates through all 9 columns
            std::cout << grid[i][j] << " "; // Prints current cell
        }
        std::cout << "\n"; // Moves result to the next line
    }
    std::cout << "\n"; // Adds a blank line after printing the complete grid
}
```

With this:

C++

```cpp
// Helper method to print any 9x9 grid to any output stream (console or file)
void displayGrid(char grid[9][9], std::ostream& out = std::cout) {
    for (int i = 0; i < 9; i++) { // This for loop iterates through all 9 rows o
        for (int j = 0; j < 9; j++) { // It will iterates through all 9 columns
            out << grid[i][j] << (j < 8 ? " " : ""); // Prints current cell with
        }
        out << "\n"; // Moves result to the next line
    }
    out << "\n"; // Adds a blank line after printing the complete grid
}
```

3. Write Each Found Solution to `outFile` in `solve()`

Location: Inside the base case (`if (row == 9)`) of the `solve(int row, int col)` method (around **Lines 77–83**).
Action: Add `displayGrid(currentGrid, outFile);` right after displaying to the console so each solution grid is written directly to `solutionN.txt` in the exact input format (without the `"Solution X:"` header in the file). CPP

Replace this:

C++

```
        if (row == 9) { // Checks if the search has reached the row after the end ro
            solutionCount++; // Increment the counter for completed solution
            std::cout << "Solution " << solutionCount << ":\n"; // Prints solution c
            displayGrid(currentGrid); // Display/prints the completed sudoku board
            return;
        }
```

With this:

C++

```
        if (row == 9) { // Checks if the search has reached the row after the end ro
            solutionCount++; // Increment the counter for completed solution
            std::cout << "Solution " << solutionCount << ":\n"; // Prints solution c
            displayGrid(currentGrid, std::cout); // Display/prints the completed sud
            displayGrid(currentGrid, outFile);   // NEW: Write completed sudoku boar
            return;
        }
```

4. Update the `SudokuGame` Constructor

Location: Right under `public:` (around **Line 119**).
Action: Update the constructor to take the output filename as a second parameter. `CPP`

Replace this:

C++

```
    // Constructor initializes the filename and solution counter
    SudokuGame(const std::string& fname) : filename(fname), solutionCount(0) {}
```

With this:

C++

```
    // Constructor initializes the input/output filenames and solution counter
    SudokuGame(const std::string& fname, const std::string& outFname)
        : filename(fname), outputFilename(outFname), solutionCount(0) {}
```

5. Open `outFile` and Write Final Summary Lines in `process()`

Location: Inside `void process()` (around **Lines 154–185**).
Action: CPP

1. Open `outFile` after loading the puzzle.
2. If `!originalPuzzleIsValid()`, write `"No solution found."` to `outFile` before returning.
3. At the end of `process()`, write either `"No solution found."` or `"Solutions found: n"` to `outFile`.

Replace `process()` with these updates (marked with `// NEW`):

C++

```
    // Method to manage the solving lifecycle for the puzzle
    void process() {
        std::cout << "=======================================\n";
        std::cout << "Puzzle File: " << filename << "\n";
        std::cout << "=======================================\n";

        if (!loadPuzzle()) { // stops puzzle if file is invalid
            std::cout << "Error: " << errorMessage << "\n\n"; // prints an error mes
            return; // stops if file not opens
        }

        // NEW: Open the solution output file
        outFile.open(outputFilename);
        if (!outFile.is_open()) {
            std::cout << "Error: Could not open output file " << outputFilename << "
            return;
        }

        std::cout << "Original Puzzle:\n";
        displayGrid(originalGrid); // shows the starting board unchanged

        if (!originalPuzzleIsValid()) { // Added - reject conflicting clues
            std::cout << "No solution found.\n\n"; // Print the message if no soluti
            outFile << "No solution found.\n";     // NEW: Write to solutionN.txt
            outFile.close();                       // NEW: Close file
            return; // Avoid any recursive search that is unnecessary
        }

        solutionCount = 0; // Reset counter to 0
        solve(0, 0); // start recursive search at first cell

        // If no solutions were found, display notification and write to file
        if (solutionCount == 0) { // Checks for solution count if it is 0
            std::cout << "No solution found.\n\n"; // prints how many solutions were
            outFile << "No solution found.\n";     // NEW: Write single line to solu
        } else {
            std::cout << "Total valid solutions found: " << solutionCount << "\n\n";
            outFile << "Solutions found: " << solutionCount << "\n"; // NEW: Write r
        }

        outFile.close(); // NEW: Close the output file when finished
    }
```

6. Pass `solutionN.txt` Filenames in `main()`

Location: Inside `int main()` (around **Lines 188–204**).
Action: Add an array (or dynamically construct the name) for `solution1.txt` through `solution5.txt` and pass it into the `SudokuGame` constructor. CPP

Replace `main()` with this:

C++

```cpp
int main() {
    // Array containing the five required puzzle file names
    std::string puzzleFiles[5] = { // Required 5 puzzle filenames
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    // NEW: Array containing the five corresponding solution file names
    std::string solutionFiles[5] = {
        "solution1.txt",
        "solution2.txt",
        "solution3.txt",
        "solution4.txt",
        "solution5.txt"
    };

    // Process each puzzle file sequentially using the object-oriented approach
    for (int i = 0; i < 5; i++) { // for loop that process all five files
        SudokuGame game(puzzleFiles[i], solutionFiles[i]); // MODIFIED: pass both pu
        game.process(); // Read validate and solve that puzzle
    }

    return 0; // terminates program
}
```

b. Qwen

Here are the specific modifications you need to make to your code. I've broken them down by section so you can apply them directly without replacing the entire file.

1. Add a private member for the output file stream

Location: Inside the `SudokuGame` class, in the `private:` section (around line 20)
Change: Add this new member variable:

cpp Copy

```cpp
    std::ofstream outputFile; // Output file stream for writing solutions
```

2. Update the constructor to open the corresponding solution file

Location: Inside the `SudokuGame` class, in the `public:` section (around line 108)
Change: Replace your existing constructor with this updated version:

cpp Copy

```cpp
    SudokuGame(const std::string& fname) : filename(fname), solutionCount(0) {
        // Derive output filename: puzzleN.txt -> solutionN.txt
        size_t dotPos = fname.find(".");
        if (dotPos != std::string::npos) {
            outputFile.open(fname.substr(0, dotPos).replace(dotPos - 7, 6, "solution") + ".txt");
        } else {
            outputFile.open("solution.txt"); // Fallback
        }
    }
```

3. Modify `displayGrid` to accept an output stream parameter

Location: Inside the `SudokuGame` class, in the `private:` section (around line 85)
Change: Update the signature and replace `std::cout` with the passed stream:

cpp Copy

```cpp
    // Helper method to print any 9x9 grid to a specified output stream
    void displayGrid(std::ostream& out, char grid[9][9]) {
        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                out << grid[i][j] << " "; // Changed from std::cout to 'out'
            }
            out << "\n"; // Changed from std::cout to 'out'
        }
        out << "\n"; // Changed from std::cout to 'out'
    }
```

4. Update `solve()` to write solutions to the output file

Location: Inside the `SudokuGame` class, in the `private:` section (around line 68)
Change: Modify the base case where a solution is found:

cpp Copy

```cpp
        // Base case: If we have successfully traversed past the last row, a solution is found
        if (row == 9) {
            solutionCount++;
            outputFile << "Solution " << solutionCount << ":\n"; // Changed to write to file
            displayGrid(outputFile, currentGrid); // Pass outputFile instead of std::cout
            return;
        }
```

5. Update `process()` to handle file output logic and cleanup

Location: Inside the `SudokuGame` class, in the `public:` section (around line 138)
Change: Replace your existing `process()` method with this updated version:

cpp Copy

```cpp
    void process() {
        // Keep console output for debugging/progress if needed, but direct solution data to file
        std::cout << "Processing " << filename << "...\n";

        if (!loadPuzzle()) {
            std::cout << "Error: " << errorMessage << "\n\n";
            return;
        }

        // Write original puzzle to the output file as requested
        outputFile << "Original Puzzle:\n";
        displayGrid(outputFile, originalGrid);

        if (!originalPuzzleIsValid()) {
            solutionCount = 0; // Ensure count is zero before handling no-solution case
            solve(0, 0); // Run solver to confirm (optional but keeps logic consistent)

            outputFile << "No solution found.\n"; // Requirement: single line if none found
        } else {
            solutionCount = 0;
            solve(0, 0);

            if (solutionCount == 0) {
                outputFile << "No solution found.\n";
            } else {
                outputFile << "Solutions found: " << solutionCount << "\n"; // Requirement: final count line
            }
        }

        outputFile.close(); // Important: close the file to flush and save data
        std::cout << "Finished processing. Results saved to corresponding solutionN.txt\n\n";
    }
```

6. (Optional but Recommended) Add a destructor to safely close files

Location: Inside the `SudokuGame` class, in the `public:` section
Change: Add this right after your constructor:

cpp Copy

```cpp
    ~SudokuGame() {
        if (outputFile.is_open()) {
            outputFile.close();
        }
    }
```

Key Implementation Notes:

1. **Filename Derivation Logic:** The string manipulation `fname.substr(0, dotPos).replace(dotPos - 7, 6, "solution")` safely converts `"puzzle1.txt"` to `"solution1.txt"`. It works for single or double-digit puzzle numbers (e.g., `puzzle1` -> `solution1`, `puzzle10` -> `solution10`).
2. **Stream Parameterization:** By changing `displayGrid` to accept `std::ostream& out`, we cleanly separate the formatting logic from the destination (`std::cout` vs `std::ofstream`). This is a best practice in C++.
3. **File Closure:** Explicitly calling `outputFile.close()` in `process()` ensures all buffered data is flushed to disk before the program moves to the next puzzle or terminates. The destructor acts as a safety net.
4. **Format Compliance:** The output format now strictly matches your requirements: original puzzle -> solutions (if any) -> `"Solutions found: n"` OR `"No solution found."` on a single line.

Apply these changes in order, and your program will correctly generate `solution1.txt` through `solution5.txt` with the exact formatting requested.

5.

a. Gemini
   i. Correctness: Valid C++ code, generates files as appropriate with correct contents.
   ii. Execution Time: Adds negligible overhead by writing to files in addition to outputting to stdout
   iii. Space Complexity: Adds additional fields that occupy a negligible amount of space, but aren't strictly necessary.
   iv. Maintainability: Modified displayGrid to work on any stream, whether cout or a file stream, and defaults to cout to keep compatibility with old code. outFile is stored in the class, which isn't as clean or proper as acquiring the handle in process and passing it where necessary. Arrays in main are hardcoded, which means both must be synchronized to avoid naming bugs. Output logic is placed in solve, which doesn't separate responsibilities appropriately.

b. Qwen

    i. Correctness: Valid C++ code, generates files, but with incorrect contents.
    ii. Execution Time: Executes solve on invalid puzzles for no reason. Negligible overhead writing to files.
    iii. Space Complexity: Only adds one additional field that occupies a negligible amount of space.
    iv. Maintainability: Uses magic numbers in the constructor of SudokuGame. Commenting “Run solver to confirm (optional but keeps logic consistent)” when running solver on invalid puzzle is misleading.
6. I choose Gemini because it generated code that did exactly what I prompted with no unnecessary changes, and the mistakes it made are much more minimal and resolvable than Qwen.
7. I removed the need for the two fields that Gemini originally wanted to place in the SudokuGame class to improve space complexity and maintainability.